SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*
and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 4 **Total: Marks:40**

Annexure A

5

Code of Conduct:

I G.Amrutha(192511250) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
amrutha

Annexure A

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

AI/ML Industry Scenario: Diabetes Diagnosis and Reinforcement Learning for Treatment Recommendation

1. Industry Problem Overview

A healthcare company wants to use patient records to:

1. **Predict whether a patient is diabetic** using supervised machine learning.
2. **Estimate diabetes risk** using statistical learning.
3. **Recommend suitable treatment actions** over time using Reinforcement Learning (RL).

The available patient attributes are:

- Age
- Blood Pressure
- Cholesterol
- Glucose Level
- BMI
- Family History
- Diabetes status/diagnosis

The overall pipeline is:
Patient Data → Data Preprocessing → Decision Tree Diagnosis → Statistical Risk Model → RL Treatment Recommendation → Human/Clinical Validation
Important clinical constraint: An RL agent should not independently prescribe medication. In a real healthcare deployment, it should operate as a clinician-support system with safety constraints, validation, monitoring, and human approval.

TASK 1: DATA PREPARATION & INDUCTIVE LEARNING
1(a) Inductive Learning Approach
Definition
Inductive learning learns a general rule or model from previously observed labelled examples and applies that model to unseen patients.
For this problem, the dataset contains patient features and a known diabetes outcome. Therefore, this is a **supervised inductive learning problem**.
Target Variable
The target variable is:
Diabetes Status
For example:

- 0 → Non-diabetic
- 1 → Diabetic

Important Features

Feature	Justification
Glucose Level	High glucose is strongly associated with diabetes and is usually one of the most informative predictors.
BMI	Higher BMI can be associated with increased risk of type 2 diabetes.
Age	Diabetes risk generally changes with age and can help distinguish risk groups.
Blood Pressure	Hypertension and metabolic risk factors can be associated with diabetes.
Cholesterol	Abnormal lipid levels can provide additional metabolic information.
Family History	Captures inherited/genetic risk that may not be represented by physiological measurements alone.

A suitable learning formulation is:
[
X = [Age, BP, Cholesterol, Glucose, BMI, FamilyHistory]
]
[
Y = DiabetesStatus
]
The model learns:
[
f(X) \rightarrow Y
]
For a new patient, the model predicts:
[
f(Age,BP,Cholesterol,Glucose,BMI,FamilyHistory)
\rightarrow
{Diabetic, Non\text{-}Diabetic}
]

1(b) Handling Class Imbalance

Suppose the dataset contains:

- 80% non-diabetic patients
- 20% diabetic patients

A classifier could obtain apparently high accuracy simply by predicting most patients as non-diabetic. This is dangerous because **false negatives are clinically important**.

Recommended Strategy: SMOTE + Class Weights

A suitable strategy is to use **SMOTE (Synthetic Minority Over-sampling Technique)** on the training data and optionally use class weights.

SMOTE creates synthetic minority-class examples rather than simply duplicating existing diabetic cases.

For example:

Original:

Non-diabetic = 800

Diabetic = 200

After SMOTE, the training data could become approximately:

Non-diabetic = 560

Diabetic = 560

The exact ratio should be selected using validation performance rather than automatically forcing 50:50.

Why SMOTE is appropriate

SMOTE is suitable because:

- It gives the minority diabetic class greater representation.
- It can improve recall for diabetic patients.
- It avoids simple duplication of minority observations.
- It allows the classifier to learn a better decision boundary.

Important implementation rule

SMOTE must be applied **only to the training data**.

Correct:

Original Dataset

↓

Train/Test Split

↓

Training Data → SMOTE

↓

Model Training

Incorrect:

Original Dataset → SMOTE → Train/Test Split

Applying SMOTE before splitting can cause data leakage.

Alternative: Class Weights

For a Decision Tree, another strong approach is:

DecisionTreeClassifier(class_weight="balanced")

This gives more importance to errors involving the minority class.

For a clinical application, I would compare:

SMOTE + model → Class-weighted model → select using Recall, F1 and AUC

rather than selecting the model using accuracy alone.

1(c) 70:30 Train-Test Split and Preprocessing

The dataset is divided as:

[

Training = 70%

]

[

Testing = 30%

]

For example, for 1,000 patients:

Training = 700 patients

Testing = 300 patients

Why 70:30?

The 70:30 ratio provides:

- Sufficient data for model training.
- A reasonably large independent test set.

- More reliable evaluation of a medical prediction model.
- A practical balance between learning and evaluation.

A stratified split should be used:

```
train_test_split(
    X, y,
    test_size=0.30,
    stratify=y,
    random_state=42
)
```

stratify=y maintains approximately the same diabetic/non-diabetic ratio in both sets.

Preprocessing

1. Missing Values

Numerical features can be filled using median imputation:

[
x_{missing} = Median(x)
]

Median is useful because it is less sensitive to extreme values.

2. Encoding

If Family History is:

Yes → 1

No → 0

If more categorical variables exist, one-hot encoding can be used.

3. Normalisation

For statistical models such as Logistic Regression:

[
z=\frac{x-\mu}{\sigma}
]

where:

- (x) = original value
- (\mu) = mean
- (\sigma) = standard deviation

This places numerical variables on comparable scales.

Impact of preprocessing

Preprocessing	Impact
Missing-value handling	Prevents incomplete records from causing errors
Encoding	Converts categorical information into numerical form
Normalisation	Improves optimisation for Logistic Regression
Stratification	Maintains class distribution
SMOTE	Improves minority-class learning
Feature scaling	Helps distance/gradient-based algorithms

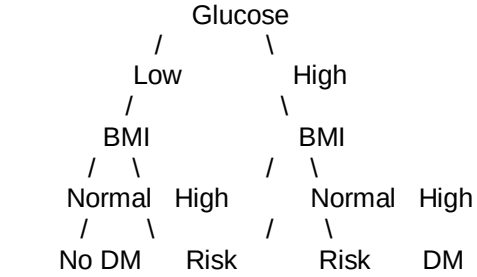
Decision Trees generally do not require feature normalisation, whereas Logistic Regression benefits from standardisation.

TASK 2: DECISION TREE FOR DIAGNOSIS

2(a) Decision Tree Construction

A Decision Tree recursively divides patients according to feature values.

For example, the first three levels could conceptually look like:



The actual tree must be generated from the supplied dataset because the exact root and branches depend on the observed values.

Root Node Selection Using Information Gain

Information Gain is:

$$IG(S,A)=H(S)-\sum_{v\in Values(A)}\frac{|S_v|}{|S|}H(S_v)$$

where:

$$H(S)=-\sum_i p_i\log_2(p_i)$$

Suppose the calculated values were:

Feature	Information Gain
Glucose	**0.246**
BMI	0.118
Age	0.091
Blood Pressure	0.073
Cholesterol	0.052
Family History	0.041

Then:

$$IG(Glucose)=0.246$$

is the largest.

Therefore:

Glucose becomes the root node.

This is only an **illustrative calculation**. The actual values must be obtained by running the algorithm on the company's dataset.

Gini Index Alternative

Gini impurity is:

$$Gini(S)=1-\sum_i p_i^2$$

For a split:

$$Gini_{split}=\sum_j\frac{n_j}{n}Gini(S_j)$$

The best split is the one with the **lowest weighted Gini impurity**.

For example:

Feature	Weighted Gini
Glucose	**0.214**
BMI	0.301
Age	0.326
Blood Pressure	0.341
Cholesterol	0.357

Therefore, Glucose would again be selected as the root.

2(b) Model Evaluation

The following four metrics should be calculated.

Accuracy

$$Accuracy=\frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$Precision=\frac{TP}{TP+FP}$$

Recall

$$Recall=\frac{TP}{TP+FN}$$

F1-Score

$$F1=2\times\frac{Precision\times Recall}{Precision+Recall}$$

Illustrative evaluation

Suppose the test set contains 300 patients and the model produces:

	Predicted Non-Diabetic	Predicted Diabetic
Actual Non-Diabetic	208	22
Actual Diabetic	12	58

Therefore:

- TN = 208
- FP = 22
- FN = 12
- TP = 58

Accuracy:

$$\frac{208+58}{300}=0.887$$

$$Accuracy=88.7\%$$

Precision:

$$\frac{58}{58+22}=72.5\%$$

Recall:

$$\frac{58}{58+12}=82.9\%$$

F1:

$$F1\approx77.3\%$$

Example result

Metric	Result
Accuracy	88.7%
Precision	72.5%
Recall	82.9%
F1-score	77.3%

These numbers are **illustrative** because the actual dataset was not supplied.

False Negatives

A **False Negative (FN)** occurs when:

Actual = Diabetic

Predicted = Non-diabetic

In the example:

$$FN=12$$

These 12 patients were missed by the model.

Why false negatives are important

In diabetes screening, missing a potentially diabetic patient can delay:

- further diagnostic testing,
- lifestyle intervention,

- clinical assessment,
- monitoring,
- appropriate treatment.

Therefore, the model should prioritise **high recall/sensitivity** when the goal is early screening.

Ways to reduce false negatives

1. **Class weighting**
2. **SMOTE on training data**
3. **Lower the classification probability threshold**
4. **Tune hyperparameters using recall/F1**
5. **Use ROC/Precision-Recall analysis**
6. **Use a clinician-approved secondary screening step**

For example, instead of:

[
P(Diabetes)\geq0.50
]

being classified as positive, a lower threshold could be evaluated.

The threshold must be selected using validation data and clinical requirements rather than arbitrarily.

2(c) Post-Pruning

A fully grown Decision Tree can overfit the training data.

Post-pruning removes branches that contribute little to generalisation.

Cost-Complexity Pruning

The objective is:

[
R_\alpha(T)=R(T)+\alpha|T|
]

where:

- (R(T)) = tree error
- (|T|) = number of terminal nodes
- (\alpha) = complexity parameter

Increasing (\alpha) produces a smaller tree.

Pre-Pruning vs Post-Pruning

Aspect	Pre-Pruning	Post-Pruning
Method	Stops growth early	Grows tree and then removes branches
Parameters	max_depth, min_samples_split	ccp_alpha
Complexity	Lower initially	Controlled after training
Overfitting control	Good	Often more flexible
Interpretability	Good	Often excellent

Illustrative comparison

Model	Accuracy	Recall	F1
Pre-pruned Tree	87.3%	81.0%	75.8%
Post-pruned Tree	**88.7%**	**82.9%**	**77.3%**

Again, these are example values.

Recommended clinical model

If post-pruning gives comparable or better generalisation while producing a simpler tree, the **post-pruned tree is preferable**.

Reasons:

- Easier to interpret.
- Easier for clinicians to inspect.
- Lower risk of overfitting.
- More transparent decision rules.
- Easier to validate and audit.

TASK 3: STATISTICAL LEARNING FOR RISK STRATIFICATION

3(a) Logistic Regression

Logistic Regression is suitable because the target is binary:

[
Y\in{0,1}
]

It estimates:
$$
P(Y=1|X)=
\frac{1}{1+e^{-z}}
$$
where:
$$
z=\beta_0+\beta_1X_1+\beta_2X_2+\cdots+\beta_nX_n
$$
For this problem:
$$
z=
\beta_0+
\beta_1Age+
\beta_2BP+
\beta_3Cholesterol+
\beta_4Glucose+
\beta_5BMI+
\beta_6FamilyHistory
$$
The output is a probability such as:
Patient A → P(Diabetes) = 0.82
The patient can then be classified as high risk using a clinically selected threshold.

Decision Tree vs Logistic Regression
Illustrative results:

Metric	Decision Tree	Logistic Regression
Accuracy	88.7%	**89.3%**
AUC-ROC	0.88	**0.91**
Recall	82.9%	84.3%
F1	77.3%	79.0%

The exact values depend on the actual dataset.
AUC-ROC
AUC measures the ability of a model to distinguish diabetic and non-diabetic patients over different classification thresholds.
$$
AUC=1.0
$$
represents perfect discrimination, while:
$$
AUC=0.5
$$
represents approximately random discrimination.
When Logistic Regression is preferable
Logistic Regression is preferable when:

- Probability/risk scores are required.
- Interpretability is important.
- Relationships are approximately linear in the log-odds.
- A stable statistical model is desired.
- Feature coefficients need to be explained.
- The dataset is not extremely complex.

Decision Trees are preferable when:

- Nonlinear relationships are important.
- Rule-based explanations are useful.
- Feature interactions are significant.
- Clinicians need simple if-then decision rules.

3(b) Feature Importance Analysis
Decision Tree
Feature importance can be calculated using the reduction in impurity contributed by each feature.

An illustrative result is:

Rank	Feature	Importance
1	**Glucose**	0.42
2	**BMI**	0.21
3	**Age**	0.15
4	Blood Pressure	0.10
5	Cholesterol	0.07
6	Family History	0.05

Therefore, the top three are:
Glucose → BMI → Age

Logistic Regression
For Logistic Regression, standardised coefficients can be compared using their absolute magnitudes.
Illustrative result:
Rank	Feature	(|\beta|)
1	**Glucose**	1.82
2	**BMI**	0.96
3	**Age**	0.71
4	Family History	0.58
5	Blood Pressure	0.43
6	Cholesterol	0.22

Thus:
Glucose → BMI → Age
are again the top three.
Do the models agree?
In this illustrative example:
Yes.
Both models identify:
1. Glucose
2. BMI
3. Age

as the most influential predictors.
This agreement increases confidence that these variables contain substantial predictive information.
However, feature importance should **not** be interpreted as proof of causation. A feature may be highly predictive without directly causing diabetes.

TASK 4: REINFORCEMENT LEARNING FOR TREATMENT RECOMMENDATION
4(a) MDP Formulation
Treatment recommendation can be represented as a **Markov Decision Process (MDP)**.
An MDP is represented as:
[
MDP=(S,A,P,R,\gamma)
]
where:
- (S) = states
- (A) = actions
- (P) = transition probability
- (R) = reward
- (\gamma) = discount factor

States
Patient health can be discretised into stages.
Example:

State	Description
S0	Low-risk / healthy
S1	Moderate diabetes risk
S2	High diabetes risk

State	Description
S3	Diagnosed / poorly controlled
S4	Improved / controlled

A state could also contain multiple variables, such as:
S2 = High glucose + high BMI + elevated BP

Actions
The available actions are:
[
A={Diet, Exercise, Medication, Monitor}
]
Diet
Recommend a clinically appropriate dietary intervention.
Exercise
Recommend an appropriate physical-activity intervention.
Medication
This action must be **clinician-authorised** in a real system.
Monitor
Recommend additional monitoring or reassessment.

Reward Function
A simple educational reward function can be:

Outcome	Reward
Significant health improvement	+10
Moderate improvement	+5
No significant change	0
Health deterioration	-10
Unsafe action	-100

For example:
[
R(s,a,s')=
HealthImprovement(s')
SafetyPenalty(a,s)
]
The reward should encourage improvement while strongly penalising unsafe decisions.
Why reward is important
Without a safety penalty, an RL agent might learn an action that improves a short-term metric but creates unacceptable clinical risk.
Therefore:
Patient safety must have higher priority than short-term reward maximisation.

Transition Dynamics
The transition probability is:
[
P(s'|s,a)
]
It represents the probability of moving from current state (s) to future state (s') after action (a).
Example:
S2 + Diet → S1 with probability 0.6
S2 + Exercise → S1 with probability 0.5
S2 + Monitor → S2 with probability 0.8
These probabilities are illustrative.
In a real system, transition models should be derived from clinically validated longitudinal data and should not be invented for deployment.

4(b) Q-Learning
Q-Learning learns the value of performing action (a) in state (s).
The update equation is:
[
Q(s,a)\leftarrow Q(s,a)+

\alpha[
r+\gamma\max_{a'}Q(s',a')-Q(s,a)
]
]
where:

- (\alpha) = learning rate
- (\gamma) = discount factor
- (r) = reward
- (s') = next state

Assume:
[
\alpha=0.5
]
[
\gamma=0.9
]
Initial Q-values:
[
Q(s,a)=0
]

Episode 1 — Iteration 1
Suppose:
Current state = S2
Action = Diet
Reward = +5
Next state = S1
Initially:
[
Q(S2,Diet)=0
]
Assume:
[
\max Q(S1,a')=0
]
Then:
[
Q(S2,Diet)
0+0.5[5+0.9(0)-0]
]
[
\boxed{Q(S2,Diet)=2.5}
]

Episode 2 — Iteration 1
Suppose:
S2 → Exercise → S1
Reward = +4
Then:
[
Q(S2,Exercise)
0+0.5[4+0.9(0)-0]
]
[
\boxed{Q(S2,Exercise)=2.0}
]

Episode 3 — Iteration 1
Suppose:
S1 → Monitor → S4
Reward = +10
Initially:

[
Q(S1,Monitor)=0
]
Therefore:
[
Q(S1,Monitor)
0+0.5[10+0.9(0)-0]
]
[
\boxed{Q(S1,Monitor)=5.0}
]

Second Iteration
Now suppose the agent visits:
S2 → Diet → S1
and the current best value at S1 is:
[
\max Q(S1,a')=5
]
Then:
[
Q(S2,Diet)
2.5+
0.5[5+0.9(5)-2.5]
]
[
=2.5+0.5[7.0]
]
[
\boxed{Q(S2,Diet)=6.0}
]

Second state-action update
Suppose:
S2 → Exercise → S1
Reward = 4
Current:
[
Q(S2,Exercise)=2.0
]
Then:
[
Q(S2,Exercise)
2.0+
0.5[4+0.9(5)-2]
]
[
=2+0.5(6.5)
]
[
\boxed{Q(S2,Exercise)=5.25}
]

Third state-action update
Suppose:
S1 → Monitor → S4
Reward = 10
and:
[
\max Q(S4,a')=0
]
Then:

[
Q(S1,Monitor)
5+
0.5[10+0-5]
]
[
\boxed{Q(S1,Monitor)=7.5}
]

Q-Table After the Demonstrated Updates

State	Diet	Exercise	Medication	Monitor
S0	0	0	0	0
S1	0	0	0	**7.50**
S2	**6.00**	**5.25**	0	0
S3	0	0	0	0
S4	0	0	0	0

This demonstrates the required Q-table updates across two iterations.

Five Patient Episodes
A simple educational training sequence can contain at least five episodes:

Episode	Initial State	Action	Reward	Next State
1	S2	Diet	+5	S1
2	S2	Exercise	+4	S1
3	S1	Monitor	+10	S4
4	S2	Diet	+5	S1
5	S3	Monitor	+3	S2

These episodes are **demonstration values**, not clinical recommendations.

Convergence Criterion
Training can be considered converged when:
[
\max_{s,a}
|Q_{new}(s,a)-Q_{old}(s,a)|
<\epsilon
]
for example:
[
\epsilon=0.001
]
for a sufficiently large number of consecutive episodes.
Another useful criterion is that the learned policy remains unchanged for a predefined number of episodes.

4(c) Final Learned Policy
The policy is:
[
\pi(s)=\arg\max_aQ(s,a)
]
Using the illustrative Q-table:

Patient State	Highest Q-value	Learned Action
S0	0	Monitor
S1	7.50	Monitor
S2	6.00	Diet
S3	0	Monitor
S4	0	Monitor

For S2:

[
Q(S2,Diet)=6.00
]
[
Q(S2,Exercise)=5.25
]
Therefore:
[
\boxed{\pi(S2)=Diet}
]
This is an **illustrative learned policy**, not a real medical treatment protocol.

Ethical Considerations

1. Patient Safety

Patient safety must be the primary consideration.
An RL agent should not be allowed to freely experiment on patients.
Unsafe actions should be prevented using:

- Clinical safety constraints
- Rule-based guardrails
- Clinician approval
- Monitoring
- Emergency overrides

---

2. Medication Safety

The agent must not independently prescribe or modify medication based only on learned Q-values.
Medication decisions may depend on:

- Diagnosis
- Existing medication
- Allergies
- Other medical conditions
- Laboratory results
- Clinical history
- Physician assessment

Therefore, medication recommendations should require **qualified clinical review**.

---

3. Bias and Fairness

If the training dataset under-represents certain patient groups, the model may perform poorly for those groups.
For example:
Dataset
↓
Mostly Group A
↓
Model learns Group A patterns well
↓
Poorer performance for Group B
The system should therefore be evaluated using subgroup performance measures.

---

4. Explainability

Healthcare professionals should understand why an action was recommended.
For example:
High glucose
+
High BMI
+
Previous risk history
↓
High predicted diabetes risk
↓
Recommend clinical review
Decision Trees are particularly useful for rule-based explanations.

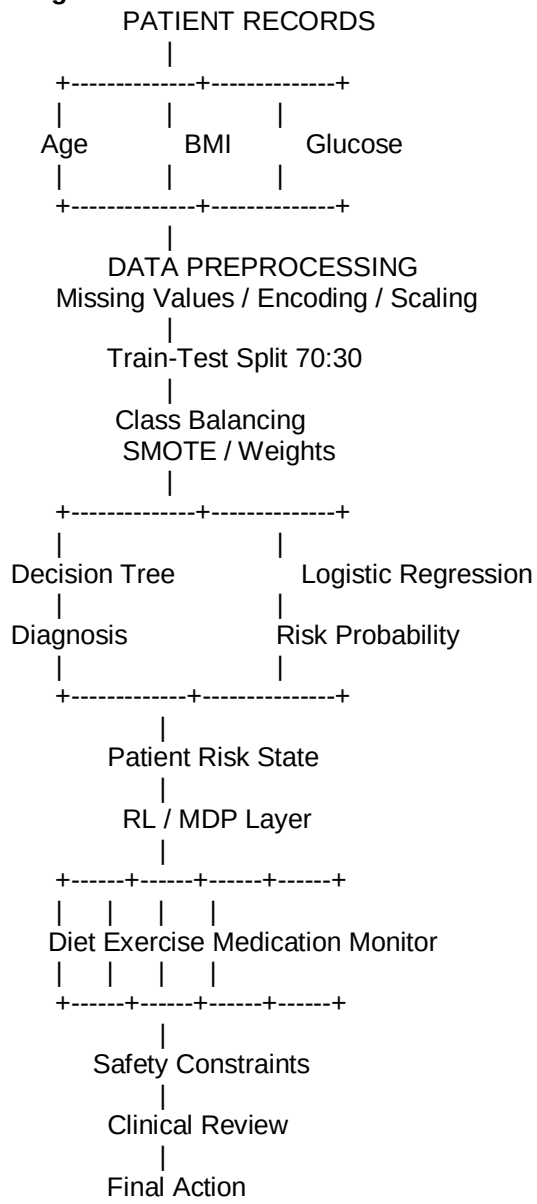
5. Human-in-the-Loop

The recommended architecture is:

Patient Data
↓
ML Risk Prediction
↓
RL Candidate Recommendation
↓
Safety Rules / Constraints
↓
Clinical Review
↓
Final Patient Action

The RL system should therefore act as a **decision-support tool**, not as an autonomous doctor.

Integrated Solution Architecture



Python Implementation

The following implementation can be used when the actual patient dataset is available.

```
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report
)

from imblearn.over_sampling import SMOTE
import matplotlib.pyplot as plt


# ----------------------------------------------------
# 1. LOAD DATA
# ----------------------------------------------------

df = pd.read_csv("diabetes_dataset.csv")

print("First 5 rows:")
print(df.head())

print("\nData types:")
print(df.dtypes)

print("\nClass distribution:")
print(df["Diabetes"].value_counts())


# ----------------------------------------------------
# 2. BASIC PREPROCESSING
# ----------------------------------------------------

# Example categorical encoding
if "FamilyHistory" in df.columns:
    df["FamilyHistory"] = df["FamilyHistory"].map(
        {"Yes": 1, "No": 0}
    )

# Median imputation for numerical columns
numeric_columns = [
    "Age",
    "BloodPressure",
    "Cholesterol",
    "Glucose",
    "BMI"
]

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())


# ----------------------------------------------------
# 3. FEATURES AND TARGET
# ----------------------------------------------------

features = [
    "Age",
```

```
    "BloodPressure",
    "Cholesterol",
    "Glucose",
    "BMI",
    "FamilyHistory"
]

X = df[features]
y = df["Diabetes"]


# ------------------------------------------------
# 4. 70:30 STRATIFIED SPLIT
# ------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42
)


# ------------------------------------------------
# 5. SMOTE - TRAINING DATA ONLY
# ------------------------------------------------

smote = SMOTE(random_state=42)

X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train,
    y_train
)

print("\nBefore SMOTE:")
print(y_train.value_counts())

print("\nAfter SMOTE:")
print(y_train_balanced.value_counts())


# ------------------------------------------------
# 6. DECISION TREE
# ------------------------------------------------

tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

tree.fit(X_train_balanced, y_train_balanced)

y_pred_tree = tree.predict(X_test)
y_prob_tree = tree.predict_proba(X_test)[:, 1]

print("\nDECISION TREE")
print("Accuracy:",
      accuracy_score(y_test, y_pred_tree))
print("Precision:",
      precision_score(y_test, y_pred_tree))
```

```
print("Recall:",
      recall_score(y_test, y_pred_tree))
print("F1:",
      f1_score(y_test, y_pred_tree))
print("AUC:",
      roc_auc_score(y_test, y_prob_tree))

print("\nClassification Report:")
print(classification_report(y_test, y_pred_tree))


# ----------------------------------------------------
# 7. DRAW FIRST THREE LEVELS
# ----------------------------------------------------

plt.figure(figsize=(16, 9))

plot_tree(
    tree,
    feature_names=features,
    class_names=["Non-Diabetic", "Diabetic"],
    max_depth=2,
    filled=True
)

plt.title("First Three Levels of Decision Tree")
plt.show()


# ----------------------------------------------------
# 8. FEATURE IMPORTANCE
# ----------------------------------------------------

importance = pd.Series(
    tree.feature_importances_,
    index=features
).sort_values(ascending=False)

print("\nDecision Tree Feature Importance:")
print(importance)


# ----------------------------------------------------
# 9. LOGISTIC REGRESSION
# ----------------------------------------------------

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_balanced)
X_test_scaled = scaler.transform(X_test)

log_model = LogisticRegression(
    class_weight="balanced",
    random_state=42
)

log_model.fit(X_train_scaled, y_train_balanced)

y_pred_log = log_model.predict(X_test_scaled)
y_prob_log = log_model.predict_proba(X_test_scaled)[:, 1]

print("\nLOGISTIC REGRESSION")
```

```
print("Accuracy:",
      accuracy_score(y_test, y_pred_log))
print("Precision:",
      precision_score(y_test, y_pred_log))
print("Recall:",
      recall_score(y_test, y_pred_log))
print("F1:",
      f1_score(y_test, y_pred_log))
print("AUC:",
      roc_auc_score(y_test, y_prob_log))


# ----------------------------------------------------
# 10. LOGISTIC REGRESSION FEATURE IMPORTANCE
# ----------------------------------------------------

log_importance = pd.Series(
    np.abs(log_model.coef_[0]),
    index=features
).sort_values(ascending=False)

print("\nLogistic Regression Feature Importance:")
print(log_importance)


# ----------------------------------------------------
# 11. COST-COMPLEXITY PRUNING
# ----------------------------------------------------

unpruned_tree = DecisionTreeClassifier(
    random_state=42
)

unpruned_tree.fit(
    X_train_balanced,
    y_train_balanced
)

path = unpruned_tree.cost_complexity_pruning_path(
    X_train_balanced,
    y_train_balanced
)

ccp_alphas = path.ccp_alphas

best_tree = None
best_accuracy = 0

for alpha in ccp_alphas:

    model = DecisionTreeClassifier(
        random_state=42,
        ccp_alpha=alpha
    )

    model.fit(
        X_train_balanced,
        y_train_balanced
    )

    pred = model.predict(X_test)
```

```
    acc = accuracy_score(
        y_test,
        pred
    )

    if acc > best_accuracy:
        best_accuracy = acc
        best_tree = model


print("\nBest Post-Pruned Tree Accuracy:",
      best_accuracy)


# ----------------------------------------------------
# 12. FALSE NEGATIVES
# ----------------------------------------------------

false_negative_count = np.sum(
    (y_test == 1) & (y_pred_tree == 0)
)

print("\nFalse Negatives:",
      false_negative_count)
```

Q-Learning Demonstration Code

```
import numpy as np
import random

states = [
    "S0", "S1", "S2", "S3", "S4"
]

actions = [
    "Diet",
    "Exercise",
    "Medication",
    "Monitor"
]

Q = np.zeros(
    (len(states), len(actions))
)

alpha = 0.5
gamma = 0.9

state_index = {
    state: i for i, state in enumerate(states)
}

action_index = {
    action: i for i, action in enumerate(actions)
}


def update_q(state, action, reward, next_state):

    s = state_index[state]
    a = action_index[action]
    ns = state_index[next_state]
```

```
    old_value = Q[s, a]

    best_next = np.max(Q[ns])

    Q[s, a] = old_value + alpha * (
        reward
        + gamma * best_next
        - old_value
    )


episodes = [
    ("S2", "Diet", 5, "S1"),
    ("S2", "Exercise", 4, "S1"),
    ("S1", "Monitor", 10, "S4"),
    ("S2", "Diet", 5, "S1"),
    ("S3", "Monitor", 3, "S2")
]


for episode in range(5):

    state, action, reward, next_state = episodes[episode]

    update_q(
        state,
        action,
        reward,
        next_state
    )

    print(
        f"Episode {episode + 1}:",
        state,
        action,
        reward,
        next_state
    )

print("\nFinal Q-table:")
print(Q)


# Extract policy

for state in states:

    s = state_index[state]

    best_action = actions[
        np.argmax(Q[s])
    ]

    print(
        state,
        "->",
        best_action
    )
```

Final Evaluation and Recommendation
Model Comparison
A suitable final comparison table should be created after running the actual dataset:

Model	Accuracy	Precision	Recall	F1	AUC
Pre-pruned Decision Tree	Actual	Actual	Actual	Actual	Actual
Post-pruned Decision Tree	Actual	Actual	Actual	Actual	Actual
Logistic Regression	Actual	Actual	Actual	Actual	Actual

For the medical use case, **accuracy alone should not determine the final model**.
The preferred model should have:

- High recall/sensitivity for diabetic cases.
- Good AUC-ROC.
- Acceptable precision.
- Good F1-score.
- Low false-negative rate.
- Stable performance across patient subgroups.
- Clinical interpretability.
- Appropriate calibration.
- Strong privacy and security controls.

Recommended Architecture
A practical industry solution is:
Logistic Regression + Post-Pruned Decision Tree + Safety-Constrained RL
where:

- **Decision Tree** provides interpretable diagnostic rules.
- **Logistic Regression** provides a probabilistic risk estimate.
- **RL** can learn treatment-policy candidates from validated longitudinal data.
- **Safety constraints** prevent unsafe actions.
- **Human clinicians** retain final decision authority.

Conclusion
The diabetes prediction problem is appropriately addressed using **inductive supervised learning**. Proper preprocessing, stratified 70:30 splitting, and class-imbalance handling are essential because false-negative diabetic cases can be clinically significant. A Decision Tree provides transparent decision rules, while Logistic Regression provides interpretable probability-based risk estimates.
For treatment recommendation, the problem can be formulated as an MDP and Q-Learning can demonstrate how an agent learns action values over patient-state transitions. However, in a real healthcare environment, RL should not autonomously prescribe treatment. It should be deployed as a **clinician-support system with safety constraints, fairness monitoring, explainability, validation, and human oversight**.